

21-Day Thin Slice Roadmap

Sprint: 21 days, started July 22, 2026 **Demo sentence (day 21):** paste an npm GitHub repo URL, scan runs with visible status, components table renders with pagination, download a valid CycloneDX file. **Team:** Dev A = frontend (React), Dev B = backend (API, queue, worker, Postgres). Lakshay = architecture, contracts, testing, demo. **Shared artifacts:** API contract (mvp-api-contract-v1.md), schema (mvp-schema-v1.sql). The contract is the single source of truth; any deviation gets fixed in the doc, not just in code.

Out of Scope (v2 backlog, do not build)

Keycloak login (mock header only). SPDX export/import and conversion. Vulnerability scanning (Grype). License compliance reporting. CERT-In 21-field report and PDF. Digital signing. Maven, pip, Go (npm only). Docker image scanning. GitLab URLs if they cost more than an hour. RBAC and user management. Deployment beyond Docker Compose.

Days 1-5: Foundations (parallel tracks)

Dev B (backend)

1. Docker Compose skeleton: Postgres, Redis, API, worker as separate services. Fallback if setup drags: Postgres/Redis in Docker, Node processes on host.
2. Apply schema; implement POST /scans and GET /scans/:id per the contract, real DB writes.
3. Worker pipeline with a placeholder scan step (timed delay standing in for Syft) walking the full status lifecycle queued through completed, plus the failure path. Proves queue and status flow before Syft complexity.

Dev A (frontend)

1. React Router with /scans/:id route. Metadata card stays as the confirm step: paste URL, see card, click Scan, POST /scans.
2. Develop against a fake API returning the contract's example responses (MSW or json-server).
3. Auto-refreshing status check with slowing refresh rate (3s, then 5s, then 10s), packaged as a reusable hook; progress UI mapped to the status enum.
4. Failed-state rendering with error message.

Lakshay: torture-test repo list, pick 3-5 demo repos, park Keycloak.

Day 5 gate: Dev B curls a scan queued through completed on the placeholder worker; Dev A runs the full flow against the fake API. Tracks independent by design.

Days 6-10: Real Pipeline

Dev B (backend)

1. Swap placeholder for real Syft: shallow clone, run Syft (cyclonedx-json output), parse with the official CycloneDX JS library, upsert components by purl, write scan_components, store raw output. Child-process timeouts on git and Syft, not just the BullMQ job. Meatiest task of the sprint.
2. Paginated GET /scans/:id/components live against real data.
3. Failure hardening, verified by breaking things on purpose: private repo fails cleanly, worker killed mid-scan recovers via stalled-job detection, hung-scan timeout fires.
4. Export v1: return the stored raw Syft output with proper headers. Deliberate shortcut, named as such; regenerating from internal tables is week 4+.

Dev A (frontend)

1. Components table against fake data, rendering the contract shape: name, version, type, licenses, direct/transitive badge; purl behind expand or copy. Total count, page 1 of N, prev/next. Client-side name filter if cheap.
2. Download button wired to export (fake file for now).
3. Empty and edge states: zero components, long names, multiple licenses.
4. Light polish only, no redesign.

Lakshay: feed Dev B a big repo and a monorepo from the torture list by day 8; 30-minute cdxgen sanity diff on one Syft output; confirm demo repo shortlist.

Day 10 milestone: Dev B demos curl-only end to end on a real npm repo: submit, statuses, real components paged, CycloneDX downloaded.

Days 11-15: Convergence + Export

Days 11-12, switchover

1. Dev A points the UI at the real API, fake API off. 2-3 days of friction (field mismatches, CORS, timing) is planned work, not slippage.
2. Both devs pair on fixes; every contract deviation gets fixed in the doc.

Dev B 3. Finish export: Content-Disposition, filename, 409 before completion. Validate one exported file with an external validator so "valid CycloneDX" is a checked claim. 4. First full-stack torture run (big repo, monorepo) through the UI. Fix breaks; log known gaps (e.g., vendored code skipped) instead of fixing everything.

Dev A 5. Real-data polish: loading states, real download flow, error states end to end. 6. No new UI features this block.

Lakshay

1. Run the full flow personally on day 13-14 as the first real user.
2. Draft the demo script, including narration for the 2-3 minute scan gap (architecture story, failed-state showcase).
3. Day 15 decision checkpoint: if export or torture fixes drag, invoke the cut line: export ships as stored raw output, torture gaps become documented known limitations.

Day 15 gate: the demo sentence works through the UI on the happy-path repos.

Days 16-21: Hardening, Buffer, Demo

Days 16-17

1. Both devs: full-flow runs on all demo repos including the deliberate failure and the big repo. One shared bug list, fixed in order: crashes, wrong data, cosmetics.
2. Dev B: seed a demo database with a completed scan as demo-day fallback.
3. Feature and contract freeze end of day 17. Bugfixes only, no exceptions.

Days 18-19 4. Deploy Compose stack on the actual demo machine: fresh clone, docker compose up, works. Doubles as first proof of the on-prem story. 5. Dev A: last cosmetic pass, bug list only. 6. Lakshay: finalize demo script with timings, rehearse scan-gap narration.

Day 20 7. Full dry run on the demo machine with a stand-in founder interrupting. One day to fix what goes wrong. 8. Complete freeze after the dry run. No commits on day 21 before the demo.

Day 21: demo. Paste URL, statuses tick, components page, CycloneDX downloads, one graceful failure shown on purpose.

Standing Rules

1. Anything not on this roadmap is v2 backlog by default.
2. Pre-agreed cut line if days 11-15 slip: export stays as stored raw output, torture gaps become known limitations, cosmetic pass is cut first. Nothing else drops before those.
3. Contract changes require both devs seeing the doc update, not just a code change.
4. Days 16-17 are the overflow buffer; the seeded database is the demo-day insurance.